C++ Module 00

Study notes — Megaphone, PhoneBook, Account

Subject version 11.2 · **Standard** C++98
Build flags `c++ -Wall -Wextra -Werror -std=c++98`
Topics namespaces, classes, member functions, stdio streams, initialisation lists, `static`, `const`

Project overview

What the module contains and how the folder is laid out.

Module 00 is the entry point to object-oriented programming at 42. Three exercises, increasing in difficulty, all restricted to the C++98 standard. The hard constraints that apply to every exercise:

- Compiles with `c++ -Wall -Wextra -Werror` and still compiles with `-std=c++98`.
- `printf()`, `malloc()` and `free()` are forbidden — grade 0 if used.
- `using namespace` and `friend` are forbidden — grade −42 if used.
- No STL containers or algorithms until Module 08. The provided `tests.cpp` is the one exception.
- No function bodies in header files (templates aside) — grade 0 for the exercise.
- Headers must be self-sufficient and carry include guards.
- Every output message ends with a newline, on standard output.

Exercise	Teaches	Files to submit
ex00 Megaphone	argv, `std::cout`, `std::string`	`Makefile`, `megaphone.cpp`
ex01 PhoneBook	classes, encapsulation, `<iomanip>`, fixed arrays	`Makefile`, `*.cpp`, `*.hpp`
ex02 Account	`static` members, `const` methods, reverse-engineering	`Makefile`, `Account.cpp`, `Account.hpp`, `tests.cpp`

Exercise 02 is **not mandatory** to pass the module, but it is the one that actually teaches `static`.

Folder layout

```
c++-0/
├── c++0.pdf                      the subject
├── ex00/  Makefile  megaphone.cpp
├── ex01/  Makefile  main.cpp  PhoneBook.hpp  PhoneBook.cpp
│                              Contact.hpp    Contact.cpp
└── ex02/  Makefile  Account.hpp  Account.cpp  tests.cpp
                    19920104_091532.log
```

In ex02, three of those four files were *given* to you: the header, the test program and the log. `Account.cpp` is the file you must reconstruct.

> **CHAPTER III OF THE SUBJECT — 42'S AI POLICY**
>
> The subject states you should not ask AI for direct answers, and that peer evaluation will require you to explain your code and perform a live modification to it. Read these notes as explanation, then write the code yourself. During the exam there is no internet and no AI.

ex00 Megaphone

Just to make sure that everybody is awake.

What the subject asks

A program that shouts its arguments back in uppercase. Given no arguments at all, it prints a fixed feedback-noise message instead.

```
$>./megaphone "shhhhh... I think the students are asleep..."
SHHHHH... I THINK THE STUDENTS ARE ASLEEP...
$>./megaphone Damnit " ! " "Sorry students, I thought this thing was off."
DAMNIT ! SORRY STUDENTS, I THOUGHT THIS THING WAS OFF.
$>./megaphone
* LOUD AND UNBEARABLE FEEDBACK NOISE *
```

THE TRAP HIDDEN IN THE EXAMPLES

The arguments are concatenated with **no separator between them**. The spaces you see around the `!` come from inside the quoted argument `" ! "` itself. If you print a space between arguments you get `DAMNIT  !  SORRY…` with doubled spaces. The subject's own general rules warn about this: "*the examples can show requirements that are not explicitly written in the instructions.*"

The implementation

```cpp
#include <iostream>
#include <string>
#include <cctype>

int main(int argc, char **argv)
{
    if (argc == 1)
    {
        std::cout << "* LOUD AND UNBEARABLE FEEDBACK NOISE *" << std::endl;
        return (0);
    }
    for (int i = 1; i < argc; ++i)
    {
        std::string arg(argv[i]);

        for (std::string::size_type j = 0; j < arg.length(); ++j)
            std::cout << static_cast<char>(std::toupper(arg[j]));
    }
    std::cout << std::endl;
    return (0);
}
```

Line by line

- **`argc == 1`** — only the program name is present, so the user passed nothing. `argc` counts `argv[0]`, so "no arguments" is 1, not 0.

- **The loop starts at `i = 1`** to skip `argv[0]`, which is the program's own name.
- **`std::string arg(argv[i])`** — converts the C string into a `std::string`. This is what the subject's "solve the exercises in a C++ manner" note is pointing at. You could walk the raw `char *` with pointers, and it would work, but the point of the exercise is to stop doing that.
- **`std::string::size_type`** rather than `int` for the index. `length()` returns an unsigned type; comparing a signed `int` against it raises `-Wsign-compare`, which `-Werror` turns into a build failure. This is the first place the module's strict flags bite.
- **`static_cast<char>`** — `std::toupper` returns an `int`, not a `char`. Streaming an `int` into `cout` prints the *number* (`72`) rather than the letter (`H`). The cast is what makes it print as a character.
- **One `std::endl` at the very end**, not one per argument — the expected output is a single line.
- **`#include <cctype>`** — `std::toupper` lives there. It often compiles without it because `<iostream>` pulls it in transitively, but relying on that is fragile and violates the subject's rule that headers must include their own dependencies.

Concepts on the syllabus here

`main`'s parameters, `std::cout` and `<<` chaining, `std::string` as a replacement for `char *`, and casts. This exercise exists mainly to confirm you can get a C++ file to compile with the required flags.

ex01 My Awesome PhoneBook

Welcome to the 80s and their unbelievable technology.

What the subject asks

An interactive phonebook. It loops reading commands and accepts exactly three: `ADD`, `SEARCH`, `EXIT`. Anything else is silently ignored. Two classes are mandatory — `PhoneBook` and `Contact` — and the phonebook holds at most 8 contacts, with a 9th replacing the oldest.

> **DYNAMIC ALLOCATION IS FORBIDDEN**
>
> No `new`. The storage must therefore be a fixed-size array declared directly inside the class: `Contact _contacts[8];`. All 8 objects exist from the moment the `PhoneBook` is constructed, whether or not the user has filled them.

Why two classes, split this way

`Contact` is a data holder: five strings and a flag. `PhoneBook` owns the array and every behaviour. The subject gives the rule for deciding what goes where — "*anything that will always be used inside a class is private, and anything that can be used outside a class is public.*"

```
class Contact
{
    public:
        Contact(void);
        ~Contact(void);
        void            set(std::string const &firstName, /* ... */);
        bool            isSet(void) const;
        std::string const   &getFirstName(void) const;
        /* ... four more getters ... */

    private:
        std::string _firstName;
        std::string _lastName;
        std::string _nickname;
        std::string _phoneNumber;
        std::string _darkestSecret;
        bool        _isSet;
};
```

The strings are private so nothing outside can reach in and rewrite `_firstName` directly; access goes through getters. This is **encapsulation**, and it is the entire point of the exercise.

Why the getters return `std::string const &`

A reference to const, for three separate reasons:

- Returning **by value** would copy the whole string on every single call.
- Returning a **non-const reference** would let the caller assign straight into the private member, defeating the encapsulation you just built.

- The function is additionally marked `const` at the end, which is a promise that calling it does not modify the object.

Why `_isSet` exists

Because a fixed array has no concept of an empty slot. All 8 `Contact` objects are default-constructed at startup, so you need some way to tell a real contact from an untouched one. *(In this implementation `PhoneBook::_count` also tracks that, making `_isSet` belt-and-braces — a legitimate design point an evaluator may question.)*

The storage: a ring buffer

```
Contact _contacts[MAX_CONTACTS];    // 8, no new, no delete
int _count;             // how many are real (caps at 8)
int _next;              // where the next ADD writes
```

```
this->_contacts[this->_next].set(firstName, lastName, nickname,
                 phoneNumber, darkestSecret);
this->_next = (this->_next + 1) % MAX_CONTACTS;
if (this->_count < MAX_CONTACTS)
    this->_count++;
```

`_next` walks 0, 1, 2 … 7 and then wraps back to 0 because of the `% 8`. On the 9th `ADD` it lands on slot 0 again and overwrites the first contact — which is exactly "replace the oldest one by the new one." That is the whole mechanism: three lines, and no shuffling of array elements.

The two counters are deliberately separate. `_count` answers *how many contacts exist*; `_next` answers *where the next one gets written*. Before the book fills they happen to hold the same value; afterwards they diverge, and confusing them is a common bug.

The display formatting

The subject requires four columns, each exactly 10 characters wide, separated by a pipe, right-aligned, and truncated with a dot as the last visible character. It also says explicitly: "*We expect you to use iomanip to reproduce the attended display.*"

```
std::cout << std::setw(10) << index << "|"
      << std::setw(10) << truncate(c.getFirstName()) << "|"
      << std::setw(10) << truncate(c.getLastName())  << "|"
      << std::setw(10) << truncate(c.getNickname())  << std::endl;
```

> **WHICH MANIPULATORS ARE STICKY**
>
> `std::setw` applies to **the next output item only** — that is why it is repeated before every column. `std::setfill` and `std::setprecision`, by contrast, **persist** until you change them again. Knowing which is which is a classic evaluation question, and it matters again in ex02.

Right alignment is `iostream`'s default under `setw`, so no explicit `std::right` is needed. The `"|"` separators are printed without a `setw`, so each takes exactly one column.

```
static std::string  truncate(std::string const &str)
{
    if (str.length() > 10)
        return (str.substr(0, 9) + ".");
    return (str);
}
```

A name of exactly 10 characters is **not** truncated — it fits. Only 11 or more gets cut. `substr(0, 9)` takes the first nine characters and `+ "."` brings it back to ten. So `Jean-Baptiste` renders as `Jean-Bapt.`

> **THREE UNRELATED MEANINGS OF ONE KEYWORD**
>
> `static` on this free function means **internal linkage** — the function is private to `PhoneBook.cpp` and invisible to the linker elsewhere. That is a completely different meaning from `static` on a class member in ex02, which means *shared by all instances*. There is a third meaning again for a `static` local variable inside a function.

Rendered result

```
     index|first name| last name|  nickname
         0|Jean-Bapt.|Chrysosto.|      Jojo
         1|        Al|        Bo|      Albo
```

Input handling

Everything reads through `std::getline(std::cin, line)`, never `std::cin >> x`.

> **THE SINGLE MOST COMMON BUG IN THIS EXERCISE**
>
> `>>` stops at the first whitespace, so a contact named "Jean Baptiste" breaks in half. Worse, `>>` leaves the trailing newline sitting in the buffer, so the *next* `getline` returns an empty string instantly without waiting for the user. Mixing the two extraction styles produces bugs that look random.

End of file (Ctrl+D)

```
if (!std::getline(std::cin, command))
    break ;
```

`getline` returns the stream, and a stream converts to `false` once it fails. Without this check, pressing Ctrl+D sends the program into an infinite loop printing the prompt forever. Evaluators test this.

Empty fields

The subject says a saved contact cannot have empty fields. `promptField` re-asks until it receives something non-empty, and returns `false` on EOF so that `add()` aborts rather than saving a half-built contact.

The index prompt

Here the subject deliberately leaves it open: "*If the index is out of range or wrong, define a relevant behavior.*" The choice made here is to accept exactly one character, require `isdigit`, and then bounds-check against

`_count`. So `"12"`, `"a"` and `""` are rejected as invalid, while `"9"` with only two contacts saved is reported as out of range. Any consistent non-crashing behaviour is acceptable — but you must be able to state which you chose and why.

Concepts on the syllabus here

Classes, public versus private, constructors and destructors, `const` member functions, references, `<iomanip>`, and fixed-size storage without allocation.

ex02 The Job Of Your Dreams

Recreating a file your colleagues deleted, because they share code on USB keys.

What the subject asks

This is a reverse-engineering exercise. You are handed three files and must reconstruct a fourth:

Given file	What it tells you
`Account.hpp`	The class declaration — every **signature**, and which members are static or private.
`tests.cpp`	A `main` that drives the class — the **order** in which things are called, and with what data.
`19920104_091532.log`	The output of the program that no longer exists — the **behaviour**, exactly.
`Account.cpp`	**Missing.** This is what you write.

The header alone is not enough, and the log alone is not enough. You need both: signatures from one, semantics from the other.

Reading the header

The static data members

```
static int  _nbAccounts;
static int  _totalAmount;
static int  _totalNbDeposits;
static int  _totalNbWithdrawals;
```

`static` on a data member means **one single variable shared by every instance of the class** — not one copy per object. There are 8 accounts but only ever one `_totalAmount`. That is precisely how `displayAccountsInfos()` can report bank-wide totals without being handed any particular account.

FOUR LINES THAT ARE EASY TO FORGET

A static member is only *declared* in the header. It must be *defined* exactly once in a translation unit, so `Account.cpp` has to open with these:

```
int Account::_nbAccounts = 0;
int Account::_totalAmount = 0;
int Account::_totalNbDeposits = 0;
int Account::_totalNbWithdrawals = 0;
```

Omit them and you get **undefined symbol** errors from the linker, not from the compiler — which is why it confuses people the first time.

The private default constructor

```
private:
    Account( void );
```

Declared, never defined, and private. Nobody can construct an `Account` without supplying an initial deposit. It is a deliberate design lock, and you do **not** implement it.

The const member functions

```
int checkAmount( void ) const;
void    displayStatus( void ) const;
```

The trailing `const` is **part of the signature**. If you drop it in your `.cpp`, you have defined a different function and the linker will report the `const` version as missing. It also matters here because `tests.cpp` reaches `displayStatus` through `std::mem_fun_ref`, which requires it.

Decoding the log

Every line is a timestamp followed by semicolon-separated `key:value` pairs. There are exactly six distinct line shapes, and each maps to one member function.

Log line	Function
`index:0;amount:42;created`	constructor
`index:0;amount:47;closed`	destructor
`accounts:8;total:20049;deposits:0;withdrawals:0`	`displayAccountsInfos`
`index:0;amount:42;deposits:0;withdrawals:0`	`displayStatus`
`index:0;p_amount:42;deposit:5;amount:47;nb_deposits:1`	`makeDeposit`
`index:0;p_amount:47;withdrawal:refused`	`makeWithdrawal` → false

1. The constructor, and where the index comes from

The first eight log lines are one per account, and their amounts — `42, 54, 957, 432, 1234, 0, 754, 16576` — are exactly the `amounts[]` array in `tests.cpp`. The indices run 0 to 7, which tells you `_accountIndex` is taken from `_nbAccounts` **before** it is incremented.

```
Account::Account(int initial_deposit) :
    _accountIndex(Account::_nbAccounts),    // 0 for the very first account
    _amount(initial_deposit),
    _nbDeposits(0),
    _nbWithdrawals(0)
{
    Account::_nbAccounts++;         // only now does it become 1
    Account::_totalAmount += initial_deposit;
    Account::_displayTimestamp();
    std::cout << "index:" << this->_accountIndex
          << ";amount:" << this->_amount
          << ";created" << std::endl;
}
```

Increment first and every index in your output is off by one.

> **INITIALISATION LISTS**
>
> The part after the colon and before the braces is the **initialisation list**. Members are constructed there directly, rather than being default-constructed and then assigned inside the body. For an `int` the difference is academic; for class-type members it saves a wasted construction, and for `const` members and references it is the *only* way to initialise them at all. The subject names initialisation lists in its own title — this is on the syllabus.

2. displayAccountsInfos

Appears three times, always immediately after the corresponding call in `tests.cpp`. The value `total:20049` is the sum of the eight initial deposits, which confirms the constructor adds to `_totalAmount`. The function is `static`, so it has no `this` pointer and can only touch static members.

3. displayStatus — and the trap next to it

```
index:0;amount:42;deposits:0;withdrawals:0
```

> **SAME KEY NAMES, DIFFERENT VARIABLES**
>
> `displayStatus` uses the key names `deposits` and `withdrawals`, exactly like `displayAccountsInfos` — but these are the **per-account** counters `_nbDeposits` and `_nbWithdrawals`, not the statics. You can prove it from the log: after all eight deposits, `displayAccountsInfos` reports `deposits:8` while every single `displayStatus` line reports `deposits:1`. Confusing the two is the most common ex02 mistake.

4. makeDeposit

```
index:0;p_amount:42;deposit:5;amount:47;nb_deposits:1
```

`p_amount` is the *previous* amount and `amount` is the value after the change, so the function must print the old balance before modifying it. It updates four things: `_amount`, `_nbDeposits`, `_totalAmount` and `_totalNbDeposits`.

5 & 6. makeWithdrawal — the key deduction of the exercise

```
index:1;p_amount:819;withdrawal:34;amount:785;nb_withdrawals:1
index:0;p_amount:47;withdrawal:refused
```

Look carefully at the refused line. It still carries `index` and `p_amount`, then says `refused`, and simply stops. That tells you the printing is **not** an if/else with two complete messages. The prefix is printed unconditionally first, and only then does the function decide:

```
Account::_displayTimestamp();
std::cout << "index:" << this->_accountIndex
      << ";p_amount:" << this->_amount
      << ";withdrawal:";            // no endl — the line is left unfinished
if (withdrawal > this->_amount)
{
    std::cout << "refused" << std::endl;
    return (false);
}
this->_amount -= withdrawal;
this->_nbWithdrawals++;
Account::_totalAmount -= withdrawal;
Account::_totalNbWithdrawals++;
std::cout << withdrawal
      << ";amount:" << this->_amount
      << ";nb_withdrawals:" << this->_nbWithdrawals << std::endl;
return (true);
```

Two accounts are refused in the log: index 0 (holds 47, asked for 321) and index 5 (holds 23, asked for 275). Both are cases where the withdrawal exceeds the balance.

Note what does *not* happen on a refusal. Those two accounts still show `withdrawals:0` in the `displayStatus` block that follows, and the bank-wide figure is `withdrawals:6`, not 8. A refused withdrawal increments nothing and changes no balance.

The timestamp

`[19920104_091532]` decodes as `YYYYMMDD_HHMMSS`, followed by one space before `index`.

```
void    Account::_displayTimestamp(void)
{
    std::time_t now = std::time(NULL);
    std::tm     *t = std::localtime(&now);

    std::cout << "[" << (t->tm_year + 1900)
          << std::setfill('0') << std::setw(2) << (t->tm_mon + 1)
          << std::setw(2) << t->tm_mday
          << "_"
          << std::setw(2) << t->tm_hour
          << std::setw(2) << t->tm_min
          << std::setw(2) << t->tm_sec
          << std::setfill(' ') << "] ";
}
```

- **Two `<ctime>` quirks:** `tm_year` counts from 1900, and `tm_mon` runs 0–11. Hence the `+ 1900` and the `+ 1`.
- **`setfill('0')` is sticky**, unlike `setw` — so it is set once and each field then only needs its own `setw(2)`. Without it, August prints as ` 8` with a leading space instead of `08`.
- **It must be reset to `' '`** at the end, or every later `setw` anywhere in the program starts padding with zeros.

Verifying against the log

Strip the timestamps from both and diff them. The first 51 lines match the 1992 log exactly. The last 8 — the destructors — are the same lines in the opposite order.

> **THIS DIFFERENCE IS EXPECTED, AND THE SUBJECT SAYS SO**
>
> *"The order in which the destructors are called may differ depending on your compiler/operating system. So your destructors may be called in a reverse order."* `std::vector` does not specify the order in which it destroys its elements; modern clang goes front-to-back, the original 1992 compiler went back-to-front.

Concepts on the syllabus here

Static data members and static member functions, `const` member functions, initialisation lists, stream manipulators, and the discipline of deriving an implementation from observed behaviour.

Preparing for the defense

The questions that actually get asked, in rough order of likelihood.

The subject warns that evaluation includes a live modification: "*a minor behaviour change, a few lines of code to write or rewrite, or an easy-to-add feature*", feasible in a few minutes. So it is not enough to have working code — you need to be able to move around inside it.

1. **What does `static` mean on `_nbAccounts`, and why are those four lines at the top of `Account.cpp`?**
   One variable shared by all instances, not one per object. Declared in the header, defined once in the .cpp — otherwise it fails at link time, not compile time.

2. **What does the `const` after `displayStatus(void)` promise?**
   That calling it does not modify the object. It is part of the signature, so dropping it in the .cpp defines a different function.

3. **Why an initialisation list instead of assignments in the constructor body?**
   Members are constructed directly rather than default-constructed then overwritten. For const members and references it is the only option.

4. **Show me the line where the 9th contact overwrites the first.**
   `_next = (_next + 1) % MAX_CONTACTS;` *— the modulo is the whole mechanism.*

5. **Does `setw` apply to everything after it, or only the next item?**
   Only the next item. `setfill` *and* `setprecision` *persist. Both facts are load-bearing in this module.*

6. **What happens if I press Ctrl+D at your prompt?**
   The stream fails, `getline` *returns false, the loop breaks. Without the check it spins forever.*

7. **Why is `Contact`'s data private if you wrote getters for all of it anyway?**
   Getters are read-only and return const references — the class controls what leaves and nothing can write in. Public members would give away both.

8. **Why can't you write `Account a;` with no argument?**
   The default constructor is declared private and never defined. It is a deliberate lock.

9. **Why does `std::cin >> name` break your phonebook?**
   It stops at whitespace and leaves the newline in the buffer, so the next getline returns empty immediately.

10. **Why is your destructor output in a different order from the log?**
    Vector element destruction order is unspecified; the subject explicitly allows this.

Things to try before the evaluation

- Delete one of the four static definitions in `Account.cpp` and read the resulting error, so you recognise a link error versus a compile error on sight.
- Remove a `const` from a member function definition and read that error too.

- Change the phonebook capacity from 8 to 4 and confirm the ring buffer still behaves — this is exactly the kind of live modification the subject describes.
- Add a sixth field to `Contact` and see how many places you have to touch. If the answer is "many", that tells you something about your design.
- Feed the phonebook a name of exactly 10 characters and one of 11, and check the truncation boundary.